ADVANCED PHASE

Day 19
Context Managers

The **with** protocol, custom context managers & contextlib

Phase	Advanced (Days 17-25)
Session Length	2 hours
Prerequisites	Days 9, 10, 11, 13, 14
Deliverables	2 concept files + 2 practice files + this guide

30-Day Python Programming Course

Instructor Session Guide

Session Overview

Goal: Students understand why context managers exist, can read and write the `with` statement confidently, can build their own class-based context managers using `__enter__`/`__exit__`, and can write faster generator-based ones with `@contextlib.contextmanager`.

Outcome: By the end of this session, students will have written at least one class-based context manager and one `@contextmanager`-based one from scratch, and will understand when Python calls `__exit__` and what its return value controls.

Session Timeline

Time	Segment	Details
0:00 - 0:10	Warm-up & recap	Quick review of Day 18 (type hints); motivate today's topic with a resource-leak bug story
0:10 - 0:35	Step 1	The problem with manual cleanup + the with statement + __enter__/__exit__ protocol
0:35 - 0:55	Step 2	Class-based context managers: exception handling in __exit__, suppressing errors
0:55 - 1:05	Break	5-10 minute breather
1:05 - 1:30	Step 3	contextlib.contextmanager decorator; generator-based context managers
1:30 - 1:50	Step 4	contextlib utilities: suppress, ExitStack, redirect_stdout, nullcontext
1:50 - 2:00	Wrap-up	Recap, mini-challenge review, assign practice problems, preview Day 20

TIP: Context managers are a conceptually dense topic for one 2-hour session. If students are struggling, it's fine to spend less time on contextlib utilities (Step 4) and let them explore those independently via the practice problems.

Prerequisites & Preparation

Requirements

Item	Requirement	Notes
Python	3.10+	Confirm with python3 --version
Editor	VS Code (from Day 1)	Pylance extension should be active
Prior knowledge	Days 9, 10, 11, 13, 14	try/except, classes, dunder methods, generators, decorators
Files ready	day19_01 / day19_02 .py files	Provided concept files for live-coding reference

Instructor Pre-Session Checklist

checkbox Read through both concept files once and run them locally to refresh timing/output

checkbox Have a real-world 'forgot to close a file/connection' anecdote ready for the opening hook

checkbox Pre-create a scratch .py file in VS Code for live-coding the class-based examples

checkbox Test that contextlib.ExitStack() and contextlib.redirect_stdout() examples run on your Python version

checkbox Print or have on-screen the 'with protocol' translation (Section 2 of concept file 1) as a reference slide

Step-by-Step Instruction Guide

STEP 1 · The with Statement & the __enter__/__exit__ Protocol

"Raise your hand if you've ever forgotten to close a file in a script. No shame -- it happens to everyone. Today we fix that forever with a Python feature called context managers. You've already been using them every time you write `with open(...) as f:` *-- today you'll learn how they work under the hood, and how to build your own."*

```
# The manual way -- works, but easy to forget:
f = open('report.txt', 'w')
try:
    f.write('some data')
finally:
    f.close()

# The 'with' way -- Python guarantees cleanup:
with open('report.txt', 'w') as f:
    f.write('some data')
# f.close() has already happened here, automatically
```

Walk through what `with EXPR as VAR:` actually expands to: `VAR = EXPR.__enter__()`, then the block body runs inside a try, and `EXPR.__exit__(exc_type, exc_value, tb)` runs in a finally -- guaranteed, even on error.

```
class Announcer:
    def __enter__(self):
        print('Entering the block...')
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print('Exiting the block...')
        return False

with Announcer() as a:
    print('Inside the block')

# Output:
# Entering the block...
# Inside the block
# Exiting the block...
```

DONE WHEN: Students can explain, in their own words, what __enter__ and __exit__ each do, and can predict the print order of a simple context manager before running it.

STEP 2 · Class-Based Context Managers & Exception Handling

"Now the important part: what happens when something goes WRONG inside a with block? This is where context managers really shine -- your __exit__ method gets told exactly what exception happened, and can decide whether to let it propagate or swallow it."

```
class DatabaseSession:
    def __enter__(self):
        print('Opening DB session...')
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        if exc_type is None:
            print('No errors -- committing.')
        else:
            print(f'Error: {exc_value} -- rolling back!')
        return False   # False = do NOT suppress the exception
```

Common errors table for this step:

Common Error	Fix / Explanation
Forgetting to return anything from __exit__	None is falsy, so this is the same as return False -- usually fine, but be explicit
Returning True unintentionally	This SILENTLY SWALLOWS every exception -- a very hard bug to track down
Confusing __exit__ args order	Signature is always (self, exc_type, exc_value, traceback) in that order

WATCH OUT: Emphasize: return True in __exit__ means 'I handled this error, don't propagate it.' This is powerful but dangerous -- an accidentally truthy return value can hide real bugs.

DONE WHEN: Students can write a class-based context manager where __exit__ conditionally suppresses one specific exception type and lets all others propagate.

STEP 3 · @contextlib.contextmanager — Generator-Based Context Managers

"Writing a whole class for something simple is a lot of typing. Python gives us a shortcut: turn a generator function into a context manager with one decorator. Everything before yield is your __enter__; everything after is your __exit__."

```
import contextlib

@contextlib.contextmanager
def db_session():
    print('Opening DB session...')
    try:
        yield {'committed': False}
    except Exception as e:
        print(f'Error: {e} -- rolling back!')
        raise
    else:
        print('No errors -- committing.')
    finally:
        print('Session closed.')
```

Command to run the live-coded example on any OS:

```
python3 day19_02_contextlib_decorator.py
```

Windows note: use `python` instead of `python3` if that's how their install is aliased (covered back on Day 1).

DONE WHEN: Students can convert one of Step 2's class-based context managers into an equivalent @contextmanager generator function, and explain why the try/finally is necessary around the yield.

STEP 4 · contextlib Utilities

"Let's speed-run four handy tools from the contextlib module that solve very common problems in one line each: suppress, ExitStack, redirect_stdout, and nullcontext."

```
# 1. suppress -- clean one-liner to ignore a specific exception
with contextlib.suppress(KeyError):
    value = some_dict['missing_key']

# 2. ExitStack -- manage an unknown/dynamic number of context managers
with contextlib.ExitStack() as stack:
    files = [stack.enter_context(open(n)) for n in filenames]

# 3. redirect_stdout -- capture printed output into a stream
buffer = io.StringIO()
with contextlib.redirect_stdout(buffer):
    noisy_function()

# 4. nullcontext -- a 'do nothing' placeholder context manager
ctx = open('log.txt', 'a') if verbose else contextlib.nullcontext()
with ctx as maybe_file:
    ...
```

TIP: These four utilities cover ~90% of real-world contextlib usage. Don't feel pressure to cover every function in the module -- suppress and ExitStack are the two students will reach for most often.

DONE WHEN: Students can pick the correct contextlib utility (suppress vs ExitStack vs nullcontext) for a given short scenario description.

Session Wrap-Up

Student Checklist

- [ ] I can explain what __enter__ and __exit__ each do, in order
- [ ] I can write a class-based context manager with proper exception handling
- [ ] I know what returning True vs False/None from __exit__ does
- [ ] I can write an equivalent context manager using @contextlib.contextmanager
- [ ] I can use contextlib.suppress() and contextlib.ExitStack() in my own code

Key Concepts Recap

Concept	What It Means
with EXPR as VAR:	Calls __enter__, binds result to VAR, guarantees __exit__ runs
__exit__ return value	True suppresses the exception; False/None lets it propagate
@contextlib.contextmanager	Turns a generator into a context manager: pre-yield = enter, post-yield = exit
contextlib.suppress()	One-line shortcut to ignore specific exception types
contextlib.ExitStack()	Manages a dynamic/unknown number of context managers safely

Troubleshooting Notes

If students report that their custom context manager isn't suppressing an exception as expected, the most common cause is forgetting to `return True` explicitly from `__exit__` -- the default return value is `None`, which does NOT suppress.

If a @contextmanager-decorated function raises `RuntimeError: generator didn't stop`, the generator has more than one `yield` statement, or code after the final yield is missing a return -- remind students each context manager function should yield exactly once.

Next Day Preview — Day 20: Testing

Tomorrow's session moves into `unittest` and `pytest`: test cases, fixtures, mocking, and coverage reports. Fun fact worth mentioning to students: pytest fixtures are themselves built on the context manager protocol from today's session!

Quick Reference Card

Daily Workflow Commands

Task	Command
Run a concept file	python3 day19_01_context_manager_protocol.py
Run the practice set	python3 day19_practise_problems_01_context_manager_protocol.py
Check Python version	python3 --version

VS Code Shortcuts

Action	Shortcut
Run current file	Ctrl+F5 / Cmd+F5 (no debugger)
Toggle terminal	Ctrl+` / Cmd+`
Format document	Shift+Alt+F / Shift+Option+F
Comment/uncomment line	Ctrl+/ / Cmd+/

Context Manager Cheat Sheet

```
# Class-based skeleton:
class MyContextManager:
    def __enter__(self):
        return self          # or another object
    def __exit__(self, exc_type, exc_value, traceback):
        return False         # True suppresses the exception

# Generator-based skeleton:
@contextlib.contextmanager
def my_context_manager():
    # setup (runs on enter)
    try:
        yield some_value
    finally:
        pass  # teardown (always runs on exit)
```

Tool Summary

Tool	Purpose
contextlib.contextmanager	Write context managers as generator functions
contextlib.suppress(*exc)	Ignore specific exception types cleanly

Tool	Purpose
contextlib.ExitStack()	Manage a dynamic number of context managers
contextlib.redirect_stdout()	Capture printed output into a stream
contextlib.nullcontext()	A placeholder context manager that does nothing